Artificial Intelligence-DA2403

GMM vs. Hidden Markov Model Report

VedaVachan Reddy-DA24B029

April 26, 2026

Dataset	3,000 voice recordings, digits 0–9, 6 speakers
Training	Speakers 1–4 (2,000 files)
Validation	Speaker 5 (500 files)
Test	Speaker 6 (500 files)
Models	GMM ($K = 12$) & Discrete HMM ($N = 7$, $M = 48$)
Features	60-dim MFCC + Δ + $\Delta\Delta$

Introduction

This report documents the design, implementation, and evaluation of two probabilistic classifiers **Gaussian Mixture Model (GMM)** and a **discrete Hidden Markov Model (HMM)** trained on a dataset of 3,000 voice recordings from six speakers each uttering the digits 0 through 9.

To solve this the system is designed in three main stages. First, useful features are extracted from the raw audio signals so that the important characteristics of speech are captured next these features are used to train acoustic models using both Gaussian Mixture Models (GMM) and Hidden Markov Models (HMM). Finally, the performance of the system is carefully evaluated using separate validation and test datasets to ensure that it works well on unseen speakers.

Dataset Overview

The dataset contains 3,000 WAV files. Each digit (0–9) was recorded by 6 different speakers, yielding 50 recordings per speaker per digit.

Table 1: Train / Validation / Test split

Split	Speakers	Files	Purpose
Training	1–4	2,000	Fit GMM and HMM models
Validation	5	500	Hyper-parameter comparison
Test	6	500	Final unseen-speaker evaluation

Feature Extraction (MFCC)

Raw audio was converted to 60-dimensional feature vectors using Mel-Frequency Cepstral Coefficients (MFCCs):

- **20 static MFCC coefficients** ($n_{\text{fft}} = 512$, hop length = 160 samples)
- **20 first-order delta coefficients** (Δ)
- **20 second-order delta-delta coefficients** ($\Delta\Delta$)

This 60-dimensional feature matrix $\mathbf{X} \in \mathbb{R}^{T\times 60}$ captures both the static spectral shape of speech and its temporal dynamics. Global mean $\boldsymbol{\mu}$ and standard deviation $\boldsymbol{\sigma}$ were computed from all training frames and each utterance was normalised as

$$\tilde{\mathbf{x}}_t = \frac{\mathbf{x}_t - \boldsymbol{\mu}}{\boldsymbol{\sigma}}, \qquad \hat{\mathbf{x}}_t = \frac{\tilde{\mathbf{x}}_t - \overline{\tilde{\mathbf{x}}}}{\text{std}(\tilde{\mathbf{X}}) + \epsilon}.$$

This two-stage normalisation reduces inter-speaker variability and channel effects.

Vector Quantisation for HMM

Discrete HMMs require observations in the form of integers to achieve this a K-Means codebook with $K = 48$ was trained on the normalized features using K-Means++ initialization and each feature vector was then assigned to its nearest codeword then converting the continuous data into a sequence of integers $O = (o_1, \ldots, o_T)$, where $o_t \in \{0, \ldots, 47\}$.

Models

Gaussian Mixture Model (GMM)

One diagonal-covariance GMM was trained per digit class $d \in \{0, \ldots, 9\}$. The class-conditional density is

$$p(\mathbf{x} \mid d) = \sum_{k=1}^{K} \pi_k^{(d)} \mathcal{N}\left(\mathbf{x}; \boldsymbol{\mu}_k^{(d)}, \boldsymbol{\Sigma}_k^{(d)}\right),$$

where $\boldsymbol{\Sigma}_k = \text{diag}(\sigma_{k,1}^2, \ldots, \sigma_{k,D}^2)$. The log-likelihood is

$$\log p(O \mid d) = \sum_{t=1}^{T} \log p(\mathbf{x}_t \mid d),$$

and inference assigns $\hat{d} = \arg\max_d \log p(O \mid d)$. Parameters are estimated via the EM algorithm.

Table 2: GMM hyper-parameters

Hyper-parameter	**Value**
Number of mixtures K	12
Covariance type	Diagonal
Initialisation	K-Means centroids (subset of codebook)
EM iterations	40 (early-stop if $\lvert\Delta\mathcal{L}\rvert < 10^{-4}$)
Variance floor	10^{-2}
Laplace smoothing	$\alpha = 0.01$ on B_{num}

Hidden Markov Model (HMM)

One left-to-right discrete HMM was trained per digit. The model is parameterised by $\lambda = (\mathbf{A}, \mathbf{B}, \boldsymbol{\pi})$:

$$\begin{aligned} \pi_i &= P(q_1 = i), \\ A_{ij} &= P(q_{t+1} = j \mid q_t = i), \\ B_{im} &= P(o_t = m \mid q_t = i), \end{aligned}$$

with the **Bakis (left-to-right) constraint**: $A_{ij} = 0$ for $j < i$ or $j > i + 1$. The sequence log-probability is computed via the forward algorithm in log-space:

$$\log \alpha_t(i) = \log \sum_j \exp\big(\log \alpha_{t-1}(j) + \log A_{ji}\big) + \log B_{i,o_t}.$$

Parameters are re-estimated by the **Baum-Welch** algorithm using the γ and ξ statistics.

Table 3: HMM hyper-parameters

Hyper-parameter	**Value**
Number of states N	7
Codebook size M	48
Topology	Left-to-right (Bakis)
Init $\boldsymbol{\pi}$	State 0 = 1.0, others = 0
Init $\mathbf{A}$	Self-loop 0.6, forward 0.4
Init $\mathbf{B}$	Uniform $(1/M)$
Baum-Welch iterations	20
Emission Laplace smoothing	add-0.01

Inference Function

Both models expose a unified inference interface. Given an audio filename:

1. Load audio and extract normalised MFCC features $\hat{\mathbf{X}}$.

2. **GMM:** $\hat{d} = \arg\max_d \log p(\hat{\mathbf{X}} \mid \lambda_d^{\text{GMM}})$.
3. **HMM:** map $\hat{\mathbf{X}}$ to VQ codes O; then $\hat{d} = \arg\max_d \log P(O \mid \lambda_d^{\text{HMM}})$.
4. Return $\hat{d}$.

Listing 1: Inference snippet (from main.py)

```
def predict(audio_file, models):
    x   = normalize(extract_mfcc(audio_file), models['mean'], models['
        std'])
    seq = models['kmeans'].predict(x)
    gmm_pred = np.argmax([models['gmm'][i].score(x)   for i in range(10)
        ])
    hmm_pred = np.argmax([models['hmm'][i].score(seq) for i in range(10)
        ])
    return gmm_pred, hmm_pred
```

Experimental Results

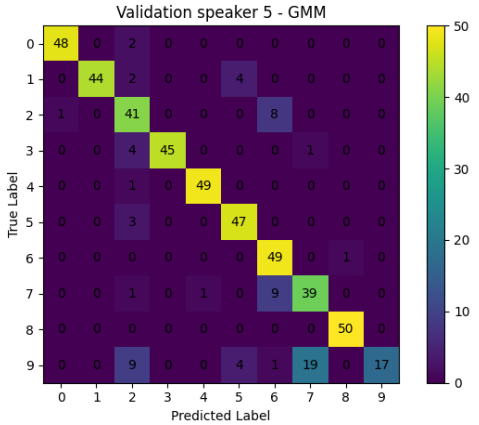
Overall Accuracy:-

Table 4: Summary of overall accuracy

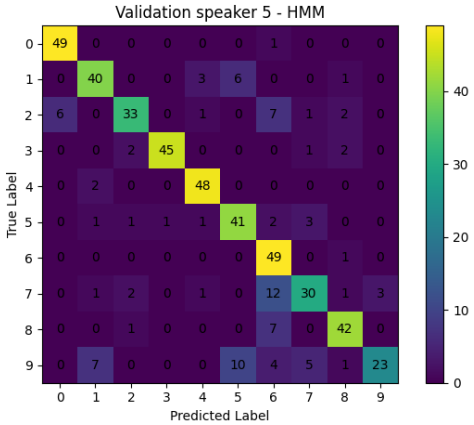
Metric	**GMM**	**HMM**
Validation accuracy (Speaker 5)	**85.8%**	80.0%
Test accuracy (Speaker 6)	**80.4%**	72.8%
Val → Test drop	−5.4 pp	−7.2 pp
Best digit — Validation	8 (100%)	0, 6 (98%)
Worst digit — Validation	9 (34%)	2 (66%)
Best digit — Test	0, 5 (100%)	1, 5 (98%)
Worst digit — Test	6 (18%)	6 (40%)

The GMM outperforms the HMM in both conditions. Both models exhibit an accuracy drop from validation to test, and this drop is larger for the HMM (−7.2 pp vs. −5.4 pp).

Confusion Matrices — Validation (Speaker 5)



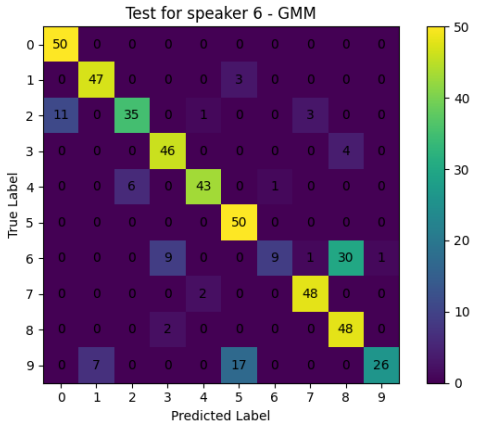
(a) GMM — Validation (Speaker 5)



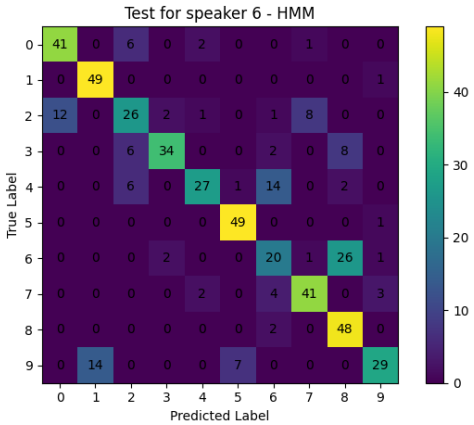
(b) HMM — Validation (Speaker 5)

Figure 1: Confusion matrices for Validation set. Rows = true digit, Columns = predicted digit.

0.1 Confusion Matrices — Test (Speaker 6)



(a) GMM — Test (Speaker 6)



(b) HMM — Test (Speaker 6)

Figure 2: Confusion matrices for Test set (Speaker 6).

Per Digit Accuracy

Table 5: Per-digit accuracy. **Bold** values indicate poor accuracy (< 0.60).

Digit	Val GMM	Val HMM	Test GMM	Test HMM
0	0.96	0.98	1.00	0.82
1	0.88	0.80	0.94	0.98
2	0.82	0.66	0.70	**0.52**
3	0.90	0.90	0.92	0.68
4	0.98	0.96	0.86	**0.54**
5	0.94	0.82	1.00	0.98
6	0.98	0.98	**0.18**	**0.40**
7	0.78	**0.60**	0.96	0.82
8	1.00	0.84	0.96	0.96
9	**0.34**	**0.46**	**0.52**	0.58
Overall	**0.858**	0.800	**0.804**	0.728

Analysis & Insights

Confusion Patterns

Digit 9 — Validation (worst for GMM)

Digit 9 is the hardest for GMM on validation (**34% accuracy**), with 19 samples misclassified as digit 7 and 9 more as digits 2 and 5. The spoken form of "nine" shares fricative and diphthong characteristics with "seven" and "five". Because the GMM treats frames as i.i.d., it conflates the bag-of-frames spectral distribution of these acoustically similar digits. The HMM (46%) partially leverages temporal sequencing to reduce this confusion.

Digit 6 — Test (catastrophic failure)

The most dramatic failure is digit 6 for Speaker 6:

- **GMM**: only 18% accuracy; **30 out of 50** samples classified as digit 8.
- **HMM**: 40% accuracy; 26 samples misclassified as digit 8, 2 as digit 3.

Phonetically, "six" and "eight" both contain a central vowel followed by fricative energy. Speaker 6's particular realisation of digit 6 maps directly to the spectral region of digit 8 in the MFCC feature space—a classic *inter-speaker phoneme overlap* problem.

Digit 2 — Cross-speaker degradation

Digit 2 degrades sharply from validation to test:

$$\text{GMM: } 82\% \to 70\%, \qquad \text{HMM: } 66\% \to 52\%.$$

Speaker 6's "two" is misclassified as "zero" (11 times under GMM), suggesting a front-vowel variant that overlaps spectrally with "zero".

Stable Digits

Digits **1, 5, 7, 8** are consistently well recognised across both models and speakers, indicating that their acoustic realisations are separable in the MFCC space regardless of speaker identity.

GMM vs. HMM

The fundamental difference between the two models is how they handle time.

GMM treats each frame independently. The utterance log-likelihood is

$$\log P(O \mid d) = \sum_{t=1}^{T} \log p(\mathbf{x}_t \mid d).$$

HMM explicitly models temporal evolution through a sequence of hidden states. The left-to-right topology assigns higher probability to sequences that follow the natural phoneme progression:

$$\log P(O \mid \lambda) = \log \alpha_T(\text{any state}).$$

Model Comparison

Comparison

Table 6: GMM vs. HMM comparison

Criterion	**GMM**	**HMM**
Validation accuracy	**85.8%**	80.0%
Test accuracy	**80.4%**	72.8%
Feature space	Continuous (60-D MFCC)	Discrete (VQ codes)
Temporal modelling	None (i.i.d. frames)	Yes (left-to-right)
Parameter count	Fewer	More (harder to estimate)
Cross-speaker robustness	Better	Worse
Catastrophic digit failures	1 digit (6 on test)	3 digits (2, 4, 6 on test)

Why GMM Wins in this Setup

1. The GMM works directly with the original continuous MFCC features, so it keeps all the details of the audio. In contrast, the HMM first converts features into discrete values (via quantisation), which leads to some loss of information.

2. **Easier to train.** The GMM has fewer parameters compared to the HMM, so it can be trained more reliably with the available data.
3. **Short audio signals.** Since each digit recording is very short (around 1–2 seconds), there is not much temporal structure for the HMM to take advantage of.
4. **Information loss in quantisation.** The K-Means step compresses the data into a limited set of codewords. This can remove subtle differences in speech, especially variations between speakers.

When HMM Would Be Preferred

In more complex speech recognition tasks, such as continuous speech or large-vocabulary problems, the order and timing of sounds become very important. In these cases, HMM-based models (such as HMM-GMM or HMM-DNN) generally perform much better than simple GMMs because they can model the sequence of speech more effectively. Therefore, the results presented in this report are specific to isolated digit recognition and should not be generalized to more complex scenarios.

finally

This report presented a complete spoken-digit recognition system using GMM and HMM classifiers implemented from scratch. Both models used 60-dimensional MFCC ($+\Delta + \Delta\Delta$) features; the HMM additionally required K-Means vector quantisation.

Key findings:

- The **GMM achieved 85.8%** validation and **80.4%** test accuracy, outperforming the HMM (80.0% and 72.8%) in both conditions.
- Digit 6 for Speaker 6 is the hardest case for both models, with GMM collapsing to 18% due to phoneme-level overlap with digit 8.
- The HMM generalises less well across speakers, primarily due to the lossy VQ quantisation step and higher parameter estimation uncertainty.
- The GMM is the preferred model for this isolated spoken-digit task.

Recommended improvements:

1. Replace discrete HMM with a **continuous-observation HMM** (diagonal Gaussian per state) to eliminate VQ information loss.
2. Increase codebook to $M \in \{64, 128\}$ and states to $N \in \{5, 9\}$, selected via cross-validation.
3. Apply **speaker normalisation** (VTLN or feature-space MLLR) before quantisation.
4. **Data augmentation**: pitch shifting (± 2 semitones) and tempo stretching ($\pm 10\%$) to improve cross-speaker robustness.
5. Explore **GMM-HMM hybrids** (GMM emission per HMM state) as the classical ASR baseline.

Overall, the GMM is the more robust and practical choice for isolated spoken-digit recognition under the given experimental conditions.